

TP CSS – Manipuler les conteneurs : float, classes, bordures arrondies et ombres

BTS SIO 1re année – Niveau 1 – Durée indicative : 3 h Contexte : agence de location de véhicules *AutoLoc Méditerranée*

1. Compétences visées

Savoir-faire techniques - Structurer une page avec des conteneurs HTML5 sémantiques (header, nav, main, section, article, aside, footer) et génériques (div). - Distinguer les éléments **de type bloc** et **en ligne**, et choisir le bon conteneur. - Placer des conteneurs **à gauche et à droite** de l'écran avec la propriété float. - Maîtriser le **flux normal** de la page et le « **débordement** » provoqué par les flottants (clear, technique du *clearfix*). - Appliquer un style à un **groupe d'éléments** grâce aux **classes** (couleurs de fond, bordures). - Utiliser le modèle de boîte : width, padding, margin, border, box-sizing. - Arrondir les angles (border-radius) et créer du relief (box-shadow). - Insérer des **images et du texte** dans des conteneurs, et faire **habiller** une image par du texte.

Savoir-être professionnels - Lire et suivre un cahier des charges, nommer clairement ses classes. - Tester et valider son travail (validateur W3C, outils de développement du navigateur).

(Ces compétences contribuent à la production de solutions applicatives web, en lien avec le bloc « Conception et développement d'applications » de l'option SLAM ; elles sont aussi utiles à l'option SISR pour la création de pages de documentation ou d'intranet.)

2. Contexte et objectif

AutoLoc Méditerranée souhaite une page de présentation de ses **agences** et de ses **offres du moment**. Vous devez réaliser cette page en plaçant les conteneurs **côte à côte** à l'aide de float.

Maquette attendue :

+-----+	
AutoLoc Méditerranée (gauche)	Accueil Agences ... (droite)
+-----+	
Nos agences	
+-----+	+-----+
Marseille	Nice
[image]	[image]
texte	texte
(bleu)	(orange pâle)
+-----+	+-----+
Nos offres du moment	
+-----+	+-----+
Renault Clio	PROMOTION
[img] texte qui habille	-20 %
l'image ...	(fond orange)
+-----+	+-----+

	+-----+	+-----+	
+	-----		+
	Pied de page		
+	-----		+

Consignes de style : 1. Conteneurs placés à **gauche** ou à **droite** avec float. 2. Chaque conteneur a une **couleur de fond** propre à sa **classe**. 3. Chaque conteneur a une **bordure aux angles arrondis**. 4. Chaque conteneur possède un **box-shadow**. 5. Chaque conteneur contient **du texte et une image**.

Méthode : pour chaque étape, lisez *l'objectif* et *l'explication*, essayez **seul**, puis comparez avec la **correction**.

3. Préparation

Créez l'arborescence suivante :

```
autoloc-float/
├── index.html
├── css/
│   └── style.css
└── images/
    ├── marseille.jpg
    ├── nice.jpg
    └── clio.jpg
```

Utilisez 3 images libres de droits d'environ 600 × 400 px. Ouvrez le dossier dans VS Code et lancez *Live Server* pour visualiser les modifications.

PARTIE A – Le HTML (structure sémantique)

Règle : aucun style dans le HTML. Les conteneurs s'empilent les uns sous les autres, c'est le **flux normal**.

Étape A1 – Le squelette HTML5

Objectif : créer un document valide relié à la feuille CSS.

Explication : un document HTML5 contient une déclaration de type, une racine <html>, un <head> (informations invisibles) et un <body> (contenu visible).

Correction :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AutoLoc Méditerranée – Nos agences et nos offres</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
```

```
</body>
</html>
```

Élément	Rôle
<!DOCTYPE html>	Indique au navigateur qu'il s'agit de HTML5
lang="fr"	Langue du contenu (accessibilité, référencement)
meta charset="UTF-8"	Encodage : affichage correct des accents
meta viewport	Adaptation à la largeur des mobiles
<title>	Titre de l'onglet du navigateur
<link rel="stylesheet">	Lie la feuille de style externe

Étape A2 – L'en-tête (logo et menu)

Objectif : créer le haut de page avec le nom de l'agence et le menu.

Explication : <header> est le conteneur d'introduction. Le logo est un simple paragraphe de classe logo. <nav> délimite la navigation, composée d'une **liste de liens**. On ajoute la classe clearfix dès maintenant : elle servira en partie B (étape B5) pour corriger un effet des flottants.

Correction (dans <body>) :

```
<header class="clearfix">
  <p class="logo">AutoLoc <span>Méditerranée</span></p>
  <nav>
    <ul>
      <li><a href="index.html">Accueil</a></li>
      <li><a href="#agences">Agences</a></li>
      <li><a href="#offres">Offres</a></li>
      <li><a href="#contact">Contact</a></li>
    </ul>
  </nav>
</header>
```

Élément	Rôle
<header>	En-tête de la page
class="clearfix"	Classe utilitaire (définie plus tard en CSS)
<p class="logo">	Texte du logo, identifié par une classe

Élément	Rôle
	Conteneur en ligne neutre, pour styliser un mot
<nav>	Zone de navigation principale
, , <a href>	Liste de liens ; #agences cible l'élément d'id="agences"

Étape A3 – Le contenu principal et la section « Nos agences »

Objectif : créer la zone principale et deux conteneurs pour les agences.

Explication : - <main> contient le contenu principal (un seul par page). - <section> regroupe un thème et porte un titre <h2>. - Chaque agence est un contenu **autonome** : <article>. - Chaque article reçoit **plusieurs classes** : bloc (style commun), bloc-marseille ou bloc-nice (couleur propre), bloc-gauche ou bloc-droite (position), moitié (largeur).

Correction (après </header>) :

```
<main>
  <h1>Nos agences et nos offres de location</h1>

  <section id="agences" class="rangee clearfix">
    <h2>Nos agences</h2>

    <article class="bloc bloc-marseille bloc-gauche moitié">
      <h3>Agence de Marseille</h3>
      
      <p>
        Située sur le Vieux-Port, notre agence principale vous
        accueille 7 jours sur 7
        de 8 h à 19 h. Plus de 80 véhicules disponibles.</p>
    </article>

    <article class="bloc bloc-nice bloc-droite moitié">
      <h3>Agence de Nice</h3>
      
      <p>
        À deux pas de la Promenade des Anglais, retrouvez nos
        citadines, cabriolets
        et utilitaires pour explorer toute la Côte d'Azur.</p>
    </article>
  </section>
</main>
```

Élément / attribut	Rôle
<main>	Contenu principal unique
<h1>	Titre principal de la page (un seul)

Élément / attribut	Rôle
<code><section id="agences"></code>	Section thématique ; id unique , cible du lien du menu
<code><h2></code> , <code><h3></code>	Hiérarchie des titres : h1 → h2 → h3
<code><article></code>	Contenu autonome (une agence)
<code>class="a b c"</code>	Un élément peut avoir plusieurs classes séparées par un espace
<code></code>	Image ; alt obligatoire (accessibilité)
<code><p></code>	Paragraphe de texte

À retenir : une **classe** (.bloc) peut être utilisée par plusieurs éléments ; un **id** (#agences) n'existe qu'une seule fois dans la page.

Étape A4 – La section « Nos offres du moment »

Objectif : créer un grand conteneur (offre) et un petit conteneur (promotion).

Explication : l'offre véhicule est un `<article>`. La promotion est un contenu **complémentaire** à la page : la balise sémantique adaptée est `<aside>`. L'image de l'offre aura une classe `img-gauche` pour que le texte s'enroule autour (étape B9).

Correction (dans `<main>`, après la première section) :

```
<section id="offres" class="rangee clearfix">
  <h2>Nos offres du moment</h2>

  <article class="bloc bloc-offre bloc-gauche large">
    <h3>Renault Clio - Citadine</h3>
    
    <p>Idéale pour se déplacer en ville et se garer facilement, la
Clio offre
      5 places, un coffre généreux et une faible consommation. Elle
est proposée
      avec kilométrage illimité et assistance 24 h/24. Réservez dès
maintenant
      pour profiter de la saison estivale sur le littoral
méditerranéen.</p>
    <p><strong>À partir de 39 € / jour</strong></p>
  </article>

  <aside class="bloc bloc-promo bloc-droite etroit">
    <h3>Promotion de la semaine</h3>
    <p class="remise">-20 %</p>
    <p>sur toute location de 7 jours et plus, du lundi au jeudi.</p>
```

```
</aside>
</section>
```

Élément / attribut	Rôle
<aside>	Contenu annexe (promotion, encart)
class="img-gauche"	Classe spécifique à l'image à habiller de texte
	Texte important (gras sémantique)
class="remise"	Classe pour styliser le pourcentage

Étape A5 – Le pied de page

Objectif : ajouter les coordonnées de l'agence.

Explication : <footer> termine la page ; <address> est réservé aux coordonnées.

Correction (après </main>) :

```
<footer id="contact">
  <address>
    AutoLoc Méditerranée – 12 quai du Port, 13002 Marseille<br>
    Tél. : 04 91 00 00 00 – contact@autoloc-med.fr
  </address>
  <p>&copy; 2026 AutoLoc Méditerranée</p>
</footer>
```

✓**Point de contrôle :** passez le code dans **validator.w3.org** : 0 erreur attendue. Ouvrez la page : tous les conteneurs sont empilés, c'est le **flux normal des éléments de type bloc**.

PARTIE B – Le CSS (conteneurs, float, couleurs, bordures, ombres)

Travaillez dans `css/style.css`. Actualisez la page après chaque étape et **observez** le résultat.

Étape B1 – Base : réinitialisation et corps de page

Objectif : partir d'une base propre et prévisible.

Explication : les navigateurs appliquent des marges par défaut différentes. * les remet à zéro. `box-sizing: border-box` fait que `width` **inclut** `padding` et `border` : très pratique pour calculer des largeurs en pourcentage.

Correction :

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
```

```

}

body {
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.6;
  color: #2b2b2b;
  background-color: #f7f3ec;
}

main {
  width: 90%;
  max-width: 1100px;
  margin: 0 auto;
  padding: 30px 0;
}

h1, h2 {
  color: #084d6b;
  margin-bottom: 20px;
}

```

Propriété	Rôle
*	Sélecteur universel : tous les éléments
margin / padding	Espace extérieur / intérieur
box-sizing: border-box	La largeur totale inclut padding et bordure
font-family	Police, avec polices de secours
line-height	Hauteur de ligne (lisibilité)
color / background-color	Couleur du texte / du fond
width: 90% + max-width	Largeur relative plafonnée
margin: 0 auto	Centre horizontalement un bloc de largeur définie
h1, h2	Sélecteur groupé

Étape B2 – Le comportement par défaut des conteneurs (observation)

Objectif : comprendre bloc et en ligne **avant** d'utiliser float.

Explication : div, section, article, p, h1... sont des éléments de **type bloc** : ils prennent toute la largeur et passent à la ligne. span, a, strong, img sont **en ligne** : ils se placent à la suite du texte. Pour placer deux blocs **côte à côte**, il faut donc changer leur comportement : c'est le rôle de float.

À faire (sans rien modifier) : ouvrez les outils de développement (**F12**), survolez <article> dans l'inspecteur et constatez qu'il occupe **toute la largeur** de <main>.

Question : pourquoi les deux agences sont-elles l'une sous l'autre ? *Réponse : ce sont des éléments de type bloc, ils occupent chacun toute la largeur disponible.*

Étape B3 – Dimensionner les conteneurs

Objectif : donner une largeur à chaque conteneur.

Explication : pour que deux conteneurs tiennent sur une même ligne, la somme de leurs largeurs doit être **inférieure à 100 %**. Ici : 46 % + 46 % = 92 % (le reste forme l'espace entre eux). Pour la rangée suivante : 62 % + 32 % = 94 %. Les largeurs sont définies par des classes **réutilisables** (moitie, large, etroit).

Correction :

```
.moitie {  
    width: 46%;  
}  
  
.large {  
    width: 62%;  
}  
  
.etroit {  
    width: 32%;  
}  
  
.bloc {  
    padding: 20px;  
    margin-bottom: 25px;  
}
```

Propriété	Rôle
.moitie	Sélecteur de classe (préfixe .)
width en %	Largeur relative à celle du parent (main)
padding: 20px	Espace intérieur entre la bordure et le contenu
margin-bottom	Espace sous le conteneur

Étape B4 – Placer les conteneurs à gauche et à droite avec float

Objectif : afficher Marseille à gauche et Nice à droite, sur la même ligne.

Explication : float: left retire l'élément du flux normal et le **pousse à gauche** ; float: right le pousse **à droite**. Les éléments suivants viennent se placer **à côté** s'il y a de la

place. On applique la propriété à deux classes de position, utilisables sur n'importe quel conteneur.

Correction :

```
.bloc-gauche {  
    float: left;  
}  
  
.bloc-droite {  
    float: right;  
}
```

Propriété	Rôle
float: left	Positionne l'élément à gauche de son parent
float: right	Positionne l'élément à droite de son parent

Observation : les agences sont côte à côte, **mais** le titre « Nos offres du moment » et le reste de la page se chevauchent ou remontent bizarrement. Passez à l'étape suivante pour comprendre pourquoi.

Étape B5 – Corriger le débordement des flottants (clearfix)

Objectif : faire en sorte que le parent « enveloppe » ses enfants flottants.

Explication : un élément flottant est **sorti du flux** : son parent (section) ne le compte plus dans sa hauteur et **s'effondre** (hauteur ≈ celle du titre seulement). La section suivante remonte alors sous les flottants. La solution : un **élément fantôme** ajouté à la fin du parent avec `::after`, qui **annule les flottants** grâce à `clear: both`. On l'applique via la classe `clearfix`, déjà posée dans le HTML.

Correction :

```
.clearfix::after {  
    content: "";  
    display: table;  
    clear: both;  
}
```

Propriété	Rôle
::after	Pseudo-élément : crée un élément virtuel après le contenu du parent
content: ""	Obligatoire pour qu'un pseudo-élément existe (ici vide)

Propriété	Rôle
<code>display: table</code>	Donne un comportement de bloc sans marge parasite
<code>clear: both</code>	Interdit tout flottant à gauche et à droite de cet élément : il se place sous les flottants

Expérience : supprimez temporairement `.clearfix::after` et observez l'effondrement de la section ; rétablissez-le ensuite.

Variantes : appliquer `clear: both` directement sur l'élément suivant, ou `overflow: hidden` sur le parent, donne aussi un résultat selon le cas, mais le `clearfix` reste la méthode la plus propre.

Étape B6 – Colorer les conteneurs par classe

Objectif : une couleur de fond différente par type de conteneur.

Explication : chaque conteneur porte deux sortes de classes : une classe **commune** (`.bloc`) pour le style partagé, et une classe **propre** (`.bloc-marseille...`) pour sa couleur. On ne répète donc jamais le même code. On colore aussi le titre `h3` de chaque conteneur grâce à un **sélecteur descendant** (`.bloc-marseille h3`).

Correction :

```
.bloc-marseille {
    background-color: #d9eef7;
}

.bloc-marseille h3 {
    color: #0b6e99;
}

.bloc-nice {
    background-color: #fde8d4;
}

.bloc-nice h3 {
    color: #c4631a;
}

.bloc-offre {
    background-color: #e3f3e8;
}

.bloc-offre h3 {
    color: #1f7a45;
}

.bloc-promo {
    background-color: #e8833a;
```

```

    color: white;
}

.bloc-promo h3 {
    color: white;
}

.remise {
    font-size: 3rem;
    font-weight: bold;
    text-align: center;
}

```

Propriété / sélecteur	Rôle
.bloc-marseille	Cible tous les éléments possédant cette classe
.bloc-marseille h3	Sélecteur descendant : les h3 situés dans ce conteneur
background-color	Couleur de fond du conteneur
color	Couleur du texte (héritée par les enfants)
font-size: 3rem	Taille relative (3 × la taille de base)
font-weight: bold	Texte en gras
text-align: center	Centre le texte dans son conteneur

Étape B7 – Ajouter une bordure aux angles arrondis

Objectif : une bordure visible et des coins arrondis sur chaque conteneur.

Explication : border est un raccourci : **épaisseur, style, couleur**. border-radius arrondit les angles : plus la valeur est grande, plus l'arrondi est important. On met la propriété sur .bloc pour tous les conteneurs, puis on ajuste la **couleur** de bordure par classe.

Correction :

```

.bloc {
    border: 2px solid #999;
    border-radius: 15px;
}

.bloc-marseille { border-color: #0b6e99; }
.bloc-nice       { border-color: #c4631a; }
.bloc-offre      { border-color: #1f7a45; }
.bloc-promo      { border-color: #a8561a; }

```

Propriété	Rôle
<code>border: 2px solid #999</code>	Raccourci : épaisseur, style (solid, dashed, dotted...), couleur
<code>border-radius: 15px</code>	Arrondit les 4 angles (accepte 1 à 4 valeurs)
<code>border-color</code>	Modifie uniquement la couleur de la bordure

Remarque : la propriété `.bloc` apparaît ici une seconde fois. En CSS, les déclarations de plusieurs règles **sur le même sélecteur se cumulent** ; si deux déclarations se contredisent, c'est la **dernière** qui l'emporte. Vous pouvez aussi regrouper toutes les déclarations de `.bloc` dans une seule règle.

Étape B8 – Ajouter une ombre portée (box-shadow)

Objectif : donner du relief aux conteneurs.

Explication : box-shadow prend : **décalage horizontal, décalage vertical, flou, couleur**. Une ombre légère (peu opaque) est plus élégante qu'une ombre noire dure. rgba permet de régler la **transparence** de la couleur.

Correction :

```
.bloc {
  box-shadow: 4px 6px 12px rgba(0, 0, 0, 0.25);
}
```

Valeur	Rôle
4px (1re)	Décalage horizontal (positif = vers la droite)
6px (2e)	Décalage vertical (positif = vers le bas)
12px (3e)	Flou : plus la valeur est grande, plus l'ombre est diffuse
rgba(0,0,0,0.25)	Noir à 25 % d'opacité (le dernier nombre va de 0 à 1)

Expériences : 1. Mettez `0 0 20px` pour une ombre uniforme autour du bloc. 2. Ajoutez `inset` au début (`inset 0 0 10px ...`) pour une ombre **intérieure**.

Étape B9 – Images et texte dans les conteneurs

Objectif : adapter les images aux conteneurs et faire habiller l'image de l'offre par le texte.

Explication : - Par défaut, une image garde sa taille d'origine et déborde. `width: 100%` la contraint à la largeur de son conteneur, `height: auto` conserve ses proportions. - `display: block` retire l'espace blanc sous l'image (élément en ligne par défaut) et permet de la séparer du texte par une marge. - Pour l'offre, on réutilise `float` à l'intérieur du conteneur : `float: left` sur l'image fait que le texte **s'enroule** autour (comme dans un magazine). `margin` crée l'espace entre l'image et le texte. - Le sélecteur `.bloc .img-gauche` est **plus précis** que `.bloc img` : il l'emporte, ce qui permet de modifier le style de cette seule image.

Correction :

```
.bloc h3 {
  margin-bottom: 12px;
}

.bloc img {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 10px;
  margin-bottom: 12px;
}

.bloc .img-gauche {
  float: left;
  width: 45%;
  margin: 0 18px 10px 0;
}

.bloc p {
  margin-bottom: 10px;
}
```

Propriété	Rôle
<code>width: 100%</code>	L'image occupe toute la largeur de son conteneur
<code>height: auto</code>	La hauteur s'ajuste pour garder les proportions
<code>display: block</code>	L'image devient un bloc (plus d'espace parasite)
<code>border-radius: 10px</code>	Arrondit aussi les angles de l'image
<code>float: left</code> (sur l'image)	L'image se place à gauche, le texte l'habille à droite
<code>margin: 0 18px 10px 0</code>	Haut 0, droite 18, bas 10, gauche 0 (sens des aiguilles d'une montre)

Propriété	Rôle
<code>.bloc .img-gauche</code>	Sélecteur descendant de classes (plus spécifique que <code>.bloc img</code>)

Observation : l'image flottante à l'intérieur de l'article est elle aussi sortie du flux. Si le texte est court, l'image peut **dépasser en bas** du conteneur. Ajoutez alors `clear: both` sur le dernier paragraphe, ou la classe `clearfix` sur l'article.

Étape B10 – L'en-tête avec float (logo à gauche, menu à droite)

Objectif : réutiliser float pour l'en-tête.

Explication : le logo est flottant à gauche, le menu à droite, comme pour les articles. L'en-tête a déjà la classe `clearfix` : il enveloppe donc ses enfants flottants. Les éléments `li` sont rendus **en ligne** avec `display: inline-block` pour former un menu horizontal.

Correction :

```
header {
  background-color: #0b6e99;
  color: white;
  padding: 15px 30px;
}
```

```
.logo {
  float: left;
  font-size: 1.5rem;
  font-weight: bold;
}
```

```
.logo span {
  color: #e8833a;
}
```

```
nav {
  float: right;
}
```

```
nav ul {
  list-style: none;
}
```

```
nav li {
  display: inline-block;
  margin-left: 20px;
}
```

```
nav a {
  color: white;
  text-decoration: none;
}
```

```
nav a:hover {
  color: #e8833a;
  text-decoration: underline;
}
```

Propriété	Rôle
float sur .logo et nav	Place le logo à gauche et le menu à droite
list-style: none	Supprime les puces
display: inline-block	li sur une même ligne, tout en gardant les marges
text-decoration: none	Supprime le soulignement des liens
a: hover	Pseudo-classe : style au survol de la souris

Étape B11 – Le pied de page

Objectif : un pied de page centré, placé sous tous les flottants.

Explication : le pied de page doit toujours apparaître **sous** les conteneurs flottants. Comme les sections sont en clearfix, c'est déjà le cas. Par sécurité, on ajoute tout de même clear: both.

Correction :

```
footer {
  clear: both;
  background-color: #084d6b;
  color: white;
  text-align: center;
  padding: 25px 20px;
}

footer address {
  font-style: normal;
  margin-bottom: 10px;
}
```

Propriété	Rôle
clear: both	Force le pied de page sous les éléments flottants
font-style: normal	Supprime l'italique par défaut de address

4. Grille de validation

Critère	OK
HTML valide au W3C (0 erreur)	☐
Balises sémantiques (header, nav, main, section, article, aside, footer)	☐
Une seule balise h1, titres hiérarchisés	☐
Marseille à gauche, Nice à droite, sur la même ligne	☐
L'offre est à gauche, la promotion à droite	☐
Chaque conteneur a sa couleur définie par sa classe	☐
Chaque conteneur a une bordure arrondie	☐
Chaque conteneur a un box-shadow	☐
Chaque conteneur contient du texte et une image (sauf la promotion : texte seul)	☐
Le texte habille correctement l'image de l'offre	☐
La page ne s'effondre pas (clearfix efficace)	☐
Aucun style dans le HTML (CSS externe uniquement)	☐

5. Pour aller plus loin

1. Ajoutez un **troisième conteneur** « Agence de Montpellier » avec une classe bloc-montpellier (couleur au choix). Comment le placer proprement avec des float ? Quelle largeur donner à chaque conteneur ?
2. Ajoutez un effet de survol sur les blocs : `.bloc:hover { box-shadow: 6px 10px 18px rgba(0,0,0,0.35); }`.
3. **Responsive** : sur écran de 700 px ou moins, faites passer les conteneurs à toute la largeur :

```
@media (max-width: 700px) {  
  .moitie, .large, .etroit {
```

```

        width: 100%;
    }
    .bloc-gauche, .bloc-droite {
        float: none;
    }
    .logo, nav {
        float: none;
        text-align: center;
    }
}

```

Expliquez le rôle de `float: none` et de `@media`. 4. **Ouverture** : aujourd'hui, la mise en page moderne utilise plutôt **Flexbox** et **CSS Grid**, et `float` est surtout conservé pour habiller une image de texte. Recherchez comment réaliser la même mise en page avec `display: flex`.

6. Questions de synthèse

1. Quelle différence entre un élément de type **bloc** et un élément **en ligne** ? Donnez trois exemples de chaque.
2. Que fait `float: left` ? Pourquoi le parent « s'effondre » ensuite ?
3. À quoi sert `clear: both` ? Expliquez la technique du **clearfix**.
4. Quelle différence entre une **classe** et un **id** ? Quand utiliser l'une ou l'autre ?
5. Que signifie `box-shadow: 4px 6px 12px rgba(0,0,0,0.25)` ? Détaillez chaque valeur.
6. Pourquoi utiliser `box-sizing: border-box` quand on travaille avec des largeurs en pourcentage ?
7. Quel est l'intérêt d'attribuer **plusieurs classes** à un même élément (`class="bloc bloc-nice bloc-droite moitie"`) ?

1. **Élément bloc** : il occupe généralement toute la largeur disponible et commence sur une nouvelle ligne.

Exemples : `<div>`, `<p>`, `<h1>`.

Élément inline : il ne crée pas de nouvelle ligne et occupe seulement la largeur nécessaire à son contenu.

Exemples : `<span>`, `<a>`, `<strong>`.

2. **float:** left fait flotter un élément vers la gauche. Les éléments suivants peuvent alors se placer à côté de lui.

Le parent peut « s'effondrer » parce qu'un élément flottant est retiré du flux normal de la page. Si le parent ne contient que des éléments flottants, sa hauteur peut devenir égale à 0.

3. **clear:** both indique qu'un élément ne doit être placé ni à côté d'un élément flottant à gauche, ni à côté d'un élément flottant à droite. Il se positionne donc sous les floats.

Le **clearfix** permet de forcer un parent à prendre en compte la hauteur de ses enfants flottants. Une technique classique consiste à utiliser un pseudo-élément :

```
.parent::after {  
  content: "";  
  display: block;  
  clear: both;  
}
```

Ainsi, le parent « englobe » correctement ses éléments flottants.

4. Une **classe** peut être utilisée sur plusieurs éléments. Elle sert à appliquer les mêmes styles à plusieurs éléments.
Un **id** est normalement unique dans une page. Il sert à identifier un élément précis.

Exemple :

```
<div class="bloc">...</div>  
<div class="bloc">...</div>  
<div id="menu">...</div>
```

On utilise donc généralement une classe pour un style réutilisable et un id pour un élément unique.

5. Chaque valeur correspond à un **paramètre** de l'ombre :

```
box-shadow: 4px 6px 12px rgba(0,0,0,0.25);
```

4px → décalage horizontal : l'ombre est décalée de 4 px vers la droite.
6px → décalage vertical : l'ombre est décalée de 6 px vers le bas.
12px → flou de l'ombre.
rgba(0,0,0,0.25) → couleur de l'ombre :
0,0,0 = noir ;
0.25 = 25 % d'opacité.

6. Avec **box-sizing: border-box**, la largeur indiquée comprend le contenu + le padding + la bordure.

Par exemple :

```
width: 50%;  
box-sizing: border-box;
```

L'élément occupe réellement 50 % de la largeur de son parent, même si on lui ajoute du padding ou une border.

Cela évite que plusieurs éléments de largeur en pourcentage dépassent la largeur disponible à cause des bordures et des espacements.

7. Un élément peut avoir plusieurs classes afin de **combiner** plusieurs styles ou comportements.

Par exemple :

```
<... class="bloc bloc-nice bloc-droite moitié">
```

On peut avoir :

- .bloc → styles généraux du bloc ;
- .bloc-nice → apparence particulière ;
- .bloc-droite → positionnement à droite ;
- .moitié → largeur de 50 %.

Cela permet de réutiliser et combiner des règles CSS sans créer une classe différente pour chaque combinaison possible.